Lean-in-the-loop RL: training setup

RL course presentation brief 11 October 2026

The question is whether reinforcement learning (RL) improves complete, unassisted number-theory proofs after supervised fine-tuning (SFT). Training is stopped. The earlier CPU engineering smoke remains isolated from future initialization. Prerequisite competence, GPU throughput, and improved proof success remain unverified.

1. Reuse assessments; prepare prerequisites

Earlier assessments exist. The saved interactive baseline failed format checks before Lean; it cannot establish capability after the parser correction. Audit parent and settings before reuse. The new frozen, no-update high-school diagnostic is prepared but unrun.

The prepared corpus uses GSM8K and MATH training splits, with separate internal development (DEV) data. Exact grouping and lexical exclusions control duplicates; they do not establish semantic independence. The order starts with arithmetic, then balances MATH subjects within competition difficulty levels. These levels are not school grades or validated prerequisites. Policy prompts contain questions; private solutions remain scorer targets with reserved context space.

Optional reinforcement as a pretraining objective (RLP) scores private solution log-likelihood gain after sampled reasoning, relative to an exponential moving average teacher without that reasoning. The warm-up is unauthorized. Likelihood gain does not certify answer correctness or transfer to Lean.

2. Separate a proof path from a training trajectory

The implemented loop in Figure 1 admits checked tactics into the proof path. Rejected actions remain outside that path. Their original token likelihoods and penalties remain in the training trajectory.

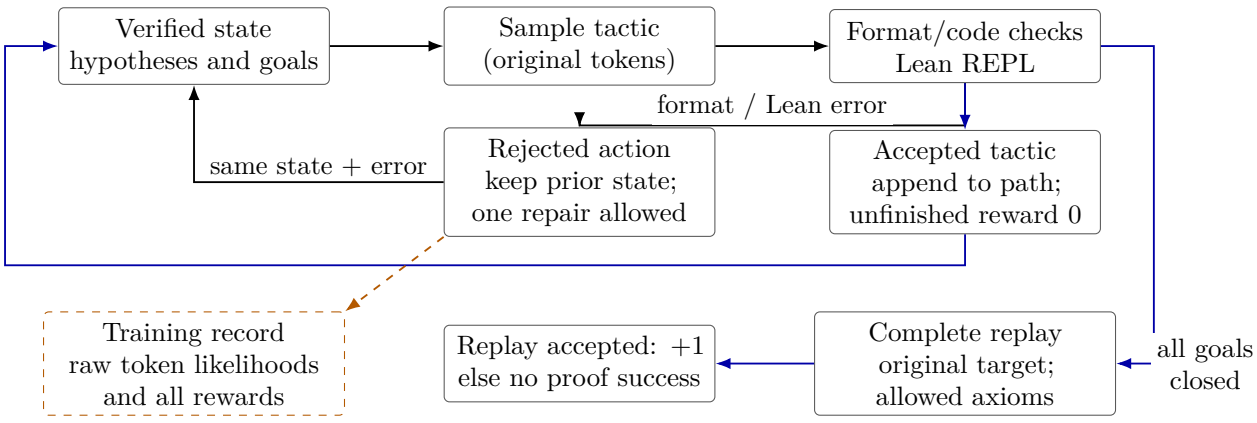


Figure 1. Blue arrows carry accepted states or complete-proof checks; the dashed arrow records rejected actions. Repairs consume existing caps. Forbidden code, repeated states, infrastructure failures, and rejected replay stop without repair.

Conditional correctness. Closing REPL goals alone is insufficient. Complete replay requires one theorem command with the original target and permits only `propext`, `Classical.choice`, and `Quot.sound`. Assuming the pinned Lean kernel and verifier checks operate correctly, acceptance certifies a proof of that formal target under those axioms. It does not prove implementation soundness, strategy, or learning.

Objective, controls, and next work

3. Use the implemented policy update

Let s_{it} be the verified goals plus any repair diagnostic for trajectory i at action t. Let a_{it} contain the original sampled tokens, including end-of-sequence when sampled. The feedback arm uses:

Reward	Outcome
$+1$	Complete replay accepted by target and axiom checks (Lean-verified).
-0.2	Format, syntax, forbidden-code, or classified unresolved-name failure (heuristic).
-0.1	Classified type/tactic failure or rejected complete replay (heuristic).
0	Valid unfinished action, repeated state, unclassified or infrastructure failure.

The terminal-only control assigns zero to every nonterminal outcome. Neither arm rewards changed goals or valid unfinished tactics. Error categories and weights are heuristic; they change the objective without an optimal-policy-preservation claim. Improved return can reflect fewer invalid actions without more proofs.

For $G \geq 2$ independent trajectories from one root, with lengths T_i and rewards r_{iu}, define

$$R_{it} = \sum_{u=t}^{T_i} r_{iu}, \qquad b_i = \frac{1}{G-1} \sum_{j \neq i} \sum_{u=1}^{T_j} r_{ju}, \qquad A_{it} = R_{it} - b_i,$$

$$L(\theta) = -\frac{1}{G} \sum_{i=1}^{G} \sum_{t=1}^{T_i} \operatorname{stopgrad}(A_{it}) \log \pi_\theta(a_{it} \mid s_{it}).$$

Here stopgrad fixes the advantage during differentiation. The update uses on-policy REINFORCE with reward-to-go and the other trajectories' mean total return as its baseline. The interactive trainer uses neither PPO nor GRPO. Likelihood sums the original token log probabilities; parsing never retokenizes the training action. Sampling and replay use the same unwarped policy with dropout disabled.

4. Hold the comparison fixed

Both prepared reward arms share the immutable common-SFT parent, task bank, seed, optimizer, retry allowance, and resource caps. Training mixes whole roots and short assisted prefixes; evaluation supplies no proof prefix. Settings are prepared, not optimized. Lean is pinned to `v4.10.0-rc1`; mathlib to `29dcec074de168ac2bf835a77ef68bbe069194c5`.

Historical SFT/RLP and SFT/GRPO interleaving includes extra SFT refreshes, preventing RL-only attribution. The prepared pair compares error feedback with terminal-only reward. Complete, unassisted proof success is the endpoint; search efficiency is deferred. Select checkpoints and task mixes on DEV only; held-out tests remain untouched. Report generated tokens, physical Lean calls, and wall time beside success. The existing `gpu_seconds` field is a CUDA wall-time proxy.

5. Follow the dependencies

1. Audit and reuse matched saved baselines. Fill unmatched coverage with the frozen, no-update diagnostic.
2. Use diagnosed weaknesses to decide whether prerequisite warm-up is needed.
3. Measure hardware throughput and memory before fixing the feasible budget.
4. After launch approval, run the matched reward comparison.
5. Evaluate both arms and the unchanged parent with identical inference budgets.

A claim that RL beats SFT also needs matched continued SFT. That control is recommended, not implemented in this pair. This document authorizes no training or compute spend.

Sources. `README.md`; `docs/EXPERIMENTS.md`, EXP-019/020.

Code: `run_tactic_rl.py`, `run_rlp_pilot.py`, `bench_lean_verifier.py`. Lean axioms; TikZ notation.